ПАРАДИГМИ ЕВОЛЮЦІОНУЮЧИХ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ

Обмеження традиційних моделей

- Традиційні моделі складності і продуктивності паралельних обчислень (наприклад, RAM або PRAM) мають характер *статичної парадигми*. Зокрема:
  - *вхідні дані* обчислень вважаються *зарані визначеними і незмінними* як за обсягом, так і за значенням;
  - *обсяг обчислень* також вважається незмінним і *визначається зарані сукупністю операцій алгоритму*, виконання яких необхідно для досягнення результату;
  - на час виконання операцій паралельного алгоритму *не накладається ніяких часових обмежень*, а важливий тільки факт їх завершення.
- Ці обмеження є, з одного боку, *спрощують аналіз* таких моделей на предмет їх складності і продуктивності обчислень.
- Проте з іншого боку, вони є *принциповими обмеженнями*, що звужують клас паралельних алгоритмів, які використовуються в системах високопродуктивних обчислень для вирішення реальних задач.

- Умови 1)-3) статичних паралельних обчислень можна називати також умовами *функціональності* обчислень.
- Справді, при обчисленні функцій в математиці та інформатиці ми завжди припускаємо, що аргументи функції при обчисленні є незмінними, а обсяг обчислень значення функції визначається тільки вхідними даними і є незмінним при заданих значеннях аргументів.
- Проте функціональні програми, як відомо, складають тільки частину мислимих обчислень, адже значна частина комп’ютерних обчислень не є функціональними.

Коментар щодо умов 1)-3)

- 1) сьогодні вже важко собі уявити паралельну обчислювальну систему, що обробляє величезні обсяги даних, які заготовлені для неї зарані і є незмінними. Скоріше в такій системі дані надходять у *вхідному потоці, певними порціями*.
- 2) -3) на час обробки таких порцій даних можуть накладатися умови реального часу, а в їх невиконання можуть наступати певні «штрафні санкції» у вигляді *втрати точності обчислень* чи *збільшення їх обсягу*.
- Такі системи паралельних обчислень називають *динамічними* або такими, що *еволюціонують* (evolvable).
- Крім зазначених вище умов 1)-3) вони можуть включати і інші аспекти динамічності моделей обчислень, наприклад, *змінну в часі обчислювальну потужність* паралельної системи.
- В останні роки моделі таких систем активно досліджувались, що призвело до нової оцінки значення паралельних обчислювальних систем і виникнення *нових парадигм паралельних обчислень.*

Проблема суперлінійної продуктивності

- Найважливішим завданням паралельних систем є *прискорення обчислень* порівняно з послідовними. Природно виникає запитання, чи може *n-процесорний* паралельний комп'ютер вирішувати одну і ту ж задачу *більш ніж в n разів швидше*, ніж на послідовному комп'ютері?
- Традиційна відповідь на це питання, загалом, заперечна (крім випадків використання прихованих ресурсів або порівняння з не найкращим послідовним алгоритмом).
- Головним аргументом тут є *принцип моделювання*, що лежить в основі поняття універсальної машини Тьюрінга (згідно з яким однопроцесорний комп'ютер може виконувати точно такі ж обчислення, що виконуються комп'ютером з *n* процесорами), а також міркування щодо функціональності традиційних обчислень: обсяг даних і обчислень – однакові для паралельного і послідовного комп’ютерів.
- Тому і *не можуть n виконувачів виконати ту саму роботу, що і для одного виконувача, швидше, ніж у n разів*.

Моделювання універсальної машини може *не мати сенсу*

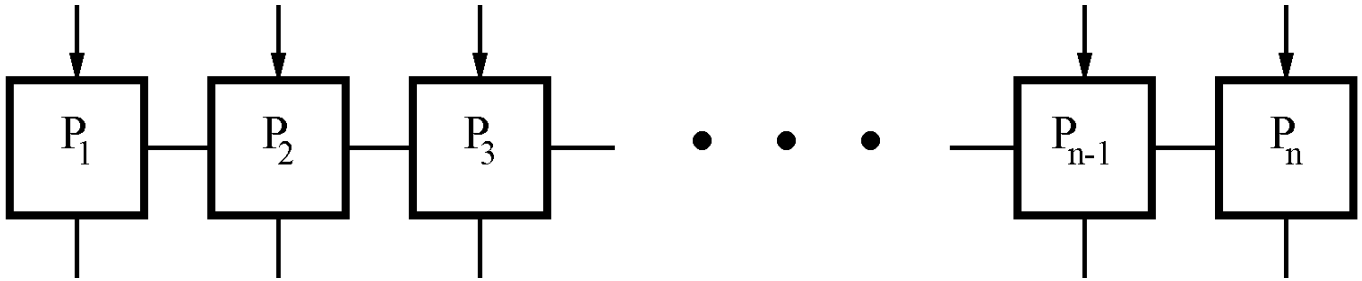
- Проте недавно виявилися кілька обчислювальних парадигм, де саме поняття моделювання універсальної машини *більше не має сенсу*.
- В межах цих парадигм, *n*-процесорний паралельний комп'ютер здатний виконувати обчислення *більш ніж у n разів швидше* (суперлінійно), ніж послідовний комп'ютер, а також, при деяких обставинах, він отримує рішення, яке є більш ніж *n* разів *кращим,* ніж те, що одержує послідовний.
- В цих випадках має місце таке явище, як *синергія* колективної роботи; паралельний комп'ютер, завдяки наявності у нього багатьох процесорів, демонструє їх синергетичну поведінку.

Послідовна модель обчислень

- Послідовна модель *S* -- це RAM-машина (1 універсальний процесор + пам’ять – набір регістрів + стрічка вводу-виводу).
- Стандартною одиницею часу в моделі *S* обчислень є час виконання *обчислювального кроку* алгоритму, тобто, виконання однієї операції.
- Протягом кожної одиниці часу обчислення процесор може:
  - отримати на вході порцію даних постійного розміру;
  - виконати певну фіксовану кількість дій (операцію) над вхідними даними;
  - передати певну порцію даних на вихід.

Паралельна модель обчислень

- Паралельна модель P (у даному випадку) є лінійний масив процесорів $P_1, P_2,\ldots, P_n$, $n\geq 2$, що з'єднуються один з одним (двосторонніми) зв'язками комунікації. Тип кожного з цих процесорів співпадає з типом процесора послідовної моделі.



- За одиницю часу кожен процесор в цій моделі може:
  - Отримувати на вході порцію даних постійного розміру від зовнішнього середовища або від іншого процесора, з яким є безпосередній зв’язок;
  - Виконувати певну фіксовану кількість дій над вхідними даними;
  - Видавати певну опрацьовану порцію даних на вихід до зовнішнього середовища або до іншого процесора, з яким є безпосередній зв’язок.

Умови обчислень реального часу

- Традиційно вважається, що всі дані, потрібні для алгоритму, *вже є доступними* для нього. Більш реалістичною є умова обчислень ***в реальному часі***, коли *не всі вхідні дані надаються зразу*, а алгоритм отримує свої дані ***протягом*** обчислень. Часто темп прибуття даних є постійним і може бути відомий наперед.
- Фундаментальною властивістю обчислень реального часу є те, що певні дії повинні виконуватись у зазначені ***крайні терміни*** (deadlines), а саме:
  - кожен елемент прийнятих *вхідних даних* повинен бути оброблений *в межах певного часу* після його прийняття.
  - кожен елемент *вихідних даних* має бути повернений *в межах певного часу* після прийняття відповідних вхідних даних.
- Тут приймається, що всі крайні терміни ***щільні,*** тобто не *можуть бути послаблені* , і що вони ***жорсткі –*** порушення крайнього терміну означає *невдачу обчислень*.

Про прискорення паралельних обчислень

- Прийнято вважати, що коефіцієнт прискорення паралельного алгоритму, що використовує ***n*** процесорів, щонайбільше дорівнює ***n***, $s_n=s(1,n)\leq n$ (так звана *'народна теорема про прискорення'*).
- Цю нерівність можна розглядати як 'погану новину', адже вона зразу накладає верхню границю на величину можливого прискорення.
- Більшість традиційних алгоритмів прикладних обчислень (як наприклад, сортування, пошуку, задач лінійної алгебри і т.д.), при їх виконанні на паралельних системах, *справді демонструють* якщо не букву, то дух цієї закономірності.

Про принцип Брента

- З іншого боку, за принципом Брента, якщо обчислення виконується на ***n*** процесорах за час $\boldsymbol{T_n}$ і на ***q*** процесорах за час $\boldsymbol{T_q}$, де $\boldsymbol{T_n < T_q}$, $2 \leq \boldsymbol{q<n}$, (використовуючи той же алгоритм), то уповільнення обчислень $d(q,n)=\boldsymbol{T_q/T_n}$. не перевищує відношення *n* до *q*, тобто, $d(q,n) \leq \lceil n/q \rceil$.
- Така нерівність – в деякому розумінні є 'доброю новиною', оскільки встановлює верхню межу на те, наскільки повільніше виконуються обчислення, коли маємо тільки ***q*** замість ***n*** процесорів.
- Більшість традиційних алгоритмів також задовольняють цю '*теорему про уповільнення*'.

Новітні результати

- Нещодавно було запропоновано кілька нетрадиційних, проте реалістичних, парадигм паралельних обчислень, що суперечать одній або обом 'теоремам' про прискорення і уповільнення:
- 1) прискорення $s(l,n) = T_l / T_n$, отримане від виконання на n процесорах у порівнянні з тим, що отримане на l процесорах, може бути суперлінійним по $\boldsymbol{n}$; а саме, можливе, наприклад, значення коефіцієнта прискорення $\boldsymbol{n^x}$, або навіть $\boldsymbol{x^n}$ для деяких $x>1$.
- 2) уповільнення $d(q,n)$ є суперлінійним по $\lceil n/q \rceil$, тому для деяких $x>1$ можна спостерігати значення коефіцієнта уповільнення $(\lceil n/q \rceil)^x$, або навіть $\boldsymbol{x}^{\lceil n/q \rceil}$.
- Отже, ці результати показують, що певні обчислення є ***суттєво паралельними.***

Парадигма накопичення даних та метафора збирання снігу

- **Метафора збирання снігу.** Якось узимку сніг падав всю ніч. Вранці сніг продовжував падати, і група з п'яти чоловік забажала вивести автомобіль з гаража. Разом вони очищають свою дорогу більше, ніж у п'ять разів швидше, ніж їх сусід, що працює сам. Щоб виїхати з двору, прибиральнику, що працює сам, доведеться прибрати снігу значно більше, ніж його п'яти сусідам, не дивлячись на той факт, що обидві ділянки дороги одного розміру.
- Отже *п'ять прибиральників зможуть забезпечити проїзд ще до того*, як ще більше нового снігу нагромадиться у них на дорозі.
- *Одному ж прибиральнику, можливо, доведеться працювати*, поки снігопад не припиниться.

Обчислювальна задача, еквівалентна метафорі прибирання снігу

- Нехай ***k*>**1 є (невелике) додатне ціле число. На вхід системи кожні ***k*** одиниць часу надходять ***n*** нових елементів даних. Над кожним з елементів даних має виконуватись операція одиничної тривалості, після чого його обробка завершується.
- Якщо ***всі отримані дані*** оброблені, то обчислення закінчується, незалежно від того, чи надійдуть ще більше даних пізніше, чи ні.
- Якщо ж на момент надходження нової порції даних обробка вже отриманих даних ще не завершена, то новий набір має стати в чергу.
- Таким чином, ***крайній термін*** для раннього закінчення обчислень складає *k*-1 одиниць часу після надходження вхідного набору.
- Існує також верхня межа $\mathbf{2^{n}}$ загальної кількості наборів ***n*** даних, які можуть бути отримані протягом даного обчислення.

Послідовний алгоритм

- Коли перша порція з ***n*** даних отримана, єдиному процесору на комп'ютері ***S*** знадобиться 1 одиниця часу для обробки кожного елемента даних, і отже вся обробка забере ***n*** одиниць часу.
- Тим часом можуть надійти $\lfloor n/k \rfloor$ додаткових наборів даних і очікувати в черзі на обробку***,*** і в такому разі комп’ютер ***S*** не встигає за прибуттям нових даних, доки вони не припинять надходити.
- Отже, ***S*** повинен обробити $2^n \times n$ значень, що, відповідно, вимагатиме $2^n \times n$ одиниць часу.

Паралельний алгоритм

- На початку обчислень, кожен з ***n*** процесорів паралельного комп’ютера ***P*** отримує один елемент від першого набору даних і обробляє його за одиницю часу.
- На початку другої одиниці часу всі доступні дані вже оброблені, і ніякі нових даних не накопичується, оскільки ***k>***1, тому паралельне обчислення закінчується.

Прискорення

- Із зазначеного вище випливає, що $T_1 = 2^n \times n$ і $\boldsymbol{T_n}$=1. Тому, ***s(1,n)=*** $2^n \times n$***,*** що значно більше, ніж максимальне прискорення ***n,*** передбачене теоремою про прискорення.
- Цей же приклад також приводить до суперечності з теоремою про уповільнення**.**

Уповільнення

- Припустимо, що паралельний комп'ютер ***P*** має тільки ***q*** процесорів, де 2 ≤ ***q <n,*** і ***(n/q) > k.***
- Перший набір даних обробляється за $\lceil n/q \rceil$ кроків ***q*** процесорами, кожен з яких обробляє щонайбільше $\lceil n/q \rceil$ даних.
- А тим часом надійдуть $\lceil n/q \rceil$***/k*** нових наборів даних і, таким чином, ***P*** не встигає обробити вхідні дані, аж поки вони не припинять надходити.
- Тому прийдеться обробити $2^n \times n$ даних, і це вимагатиме $T_q = \lceil (2^n \times n)/q \rceil$ одиниць часу.
- В результаті, уповільнення складе величину ***d(q,n)=*** $\lceil (2^n \times n)/q \rceil$, яке асимптотично набагато більше, ніж уповільнення $\lceil n/q \rceil$, передбачене теоремою Брента.

Парадигма належного вибору набору даних

- У цій моделі вирішення обчислювальної проблеми на послідовному комп’ютері ***S*** є більш, ніж у ***n*** разів повільнішим, ніж на паралельному комп’ютері ***P,*** тому що за необхідності вибору з певного набору даних, послідовна машина у найгіршому випадку *робить неправильний вибір*.
- **Метафора переслідування на колі.** Нехай деякий об’єкт ***A*** переслідує на колі об’єкт ***B. A*** і ***B*** рухаються з однаковою швидкістю; ясно, що ***A ніколи*** не наздожене ***B***. Якщо ж два об'єкти ***X*** і ***Y*** переслідують об'єкт ***B*** на колі круга, але ***X*** і ***Y*** рухаються назустріч одне одому; отже вони ***завжди*** схоплять ***B***.

Обчислювальна парадигма для переслідування та ухилення на колі

- Узагальнимо ідею на багато потоків. У цій парадигмі є ***n*** джерел даних, що діють одночасно, і забезпечують кожен потік з ***n*** даних - один елемент даних за одиницю часу.
- ***n*** даних, що містяться в будь-якому з ***n*** потоків, необхідно і достатньо для вирішення певної проблеми. Проте, тільки якийсь один потік, невідомий заздалегідь, надає дані для вирішення проблеми за ***n*** одиниць часу, а решта ***n***-1 потоків забезпечують вирішення задачі за 2^n одиниць.
- Всі потоки є однаково ймовірними щодо даних, що ведуть до рішення за ***n*** кроків.

- Крім того, дані у всіх ***n*** потоках задовольняють наступні умови:
- 1) Всі потоки різні: ніякі два потоки не містять один і той же елемент даних, і ніяка підмножина одного потоку не може комбінуватися з підмножиною іншого потоку для вирішення даної задачі.
- 2) Є обмеження крайнього терміну: обробка даних у потоці повинна початися ***щонайпізніше*** тоді, коли надійде останній елемент даних потоку. Інакше кажучи, ***n*** даних цього потоку *відживають* (стають застарілими), оскільки їх замінюють ***n*** нових значень в буфері фіксованого розміру.

Послідовний алгоритм

- Комп’ютер ***S*** випадково вибирає ***один*** з ***n*** потоків і використовує ***n*** його даних для вирішення задачі.
- У найгіршому випадку, ***S*** вибирає один з тих ***n***-1 потоків, який вимагає 2^n кроків обчислень для одержання рішення. Цей якнайгірший випадок відбувається з ймовірністю (***n***-1)/***n***, що прямує до 1 зі збільшенням ***n***.
- Отже, з великою мірою упевненості час виконання на послідовному комп’ютері ***S*** складає $T_1 = n+2^n$ кроків.

Паралельний алгоритм

- На комп’ютері ***P*** кожен з ***n*** процесорів контролює один з ***n*** потоків і намагається вирішити обчислювальну задачу, використовуючи ***n*** елементів даних цього потоку.
- Один (і тільки один) з процесорів отримує рішення за ***n*** кроків після надходження ***n***-го елемента даних у вибраному потоці.
- Цей процесор потім посилає повідомлення через лінійний масив, щоб інформувати решту n-1 процесорів, щоб *зупинити їх обчислення*.
- Обчислення на ***P*** вимагатиме $T_n = 3n-1$ кроків у найгіршому випадку (n – на прийняття даних, n – на обчислення, n-1 – на передачу повідомлень для припинення обчислень іншими процесорами).

Прискорення та уповільнення

- У найгіршому випадку прискорення складає ***s(1,n)*** = $(n+2^n)/(3n-1)$, що є суперлінійним по ***n***, і очевидно, теорема про прискорення знову не справджується.
- Якщо ***P*** матиме тільки q процесорів, де $2 \le q < n$, то з ***n*** можливих потоків q процесорів випадково вибирають q потоків. У найгіршому випадку, жоден з процесорів не вибирає той потік, де дані приводять до рішення за лінійний час в ***n*** кроків.
- Цей найгірший випадок відбувається з ймовірністю 1 - ***(q/n).*** Тому, якщо ***n*** велике, а q – маленьке, час виконання задачі на P майже напевно складе $\boldsymbol{T_q = n+2^n}$ кроків, а уповільнення ***d(q,n)***=$(n+2^n)/(3n-1)$.
- Це уповільнення є суперлінійним по $\lceil n/q \rceil$ і не залежить від ***q***, що знову порушує теорему про уповільнення.

Парадигма складної оберненої функції і метафора пересування меблів

- Третій приклад моделі для суперлінійного прискорення має справу з ситуацією, коли послідовний алгоритм більш, ніж в ***n*** разів повільніший за паралельний, тому що виконує ***непотрібні обчислення*** або ***повинен дочекатися даних***.
- **Метафора пересування меблів.** Нехай потрібно перемістити великі меблі з одного місця на інше. Один робітник, що працює сам, не в змозі підняти, штовхати або тягнути меблі і для того, щоб їх перемістити, мусить їх розібрати на частини, і перевезти кожну частину окремо, а потім зібрати їх на новому місці. Робота вимагає 1 годину часу. З іншого боку, чотири робітники, що працюють разом, можуть просто підняти меблі і помістити їх на нове місце за 15 секунд, що у 240 разів швидше.

Незворотні функції

- Функція f називається ***незворотною,*** якщо складність обчислень її інверсії f^{-1} набагато вища, ніж складність обчислень для f.
- Зокрема, алгоритму обчислення f^{-1} може просто не існувати. Наприклад, легко порахувати суму множини цілих чисел x_1, $x_2,\ldots,x_n$, проте невідомий ефективний алгоритм, який би визначав би множину цілих чисел по заданій їх сумі.
- Нехай треба порахувати значення функції $\boldsymbol{g(}x_1, x_2,\ldots,x_n\boldsymbol{)}$, і значення $\boldsymbol{g}$ обчислюється за $\boldsymbol{n}$ кроків.
- Вхідні значення $x_1, x_2,\ldots,x_n$, необхідні для обчислення g, надходять як n пар виду $(x_i, \boldsymbol{f(}x_1, x_2,\ldots,x_n\boldsymbol{)})$, для і = 1,2..., n, де обчислення $f(x_1, x_2,\ldots,x_n)$ виконується за $\boldsymbol{n}$ кроків; а зворотна функція, тобто визначення x_i із значення $f(x_1, x_2,\ldots,x_n)$, займає 2^n одиниць часу.
- $\boldsymbol{g}$ має обчислюватись ***у реальному часі***: якщо пара не обробляється за один крок після її надходження, вона застаріває (на її місце переписуються нові дані того ж розміру).

Послідовний алгоритм

- На обробку надходять одночасно ***n*** пар і запам'ятовуються в буфері. Пара $(x_1, f(x_1, x_2,...,x_n))$ читається першою в *S.* У цей момент інші (n-1) пар ще не доступні.
- Для того, щоб відновити значення $x_2, x_3,...,x_n$ послідовному процесору потрібно інвертувати *f.* Це забере $(n\text{-}1)2^n$ одиниць часу. А вже потім обчислюється $g(x_1, x_2,...,x_n)$.
- Отже, $T_1 = 1 + (n\text{-}1)2^n + (n\text{-}1)$ одиниць часу.

Паралельний алгоритм

- Як тільки ***n*** пар отримані, паралельний комп'ютер їх обробляє негайно. Процесор $\boldsymbol{P_i}$ читає пару $(x_i, \boldsymbol{f(}x_1, x_2,\ldots,x_n\boldsymbol{))}$ і обробляє x_i для $\boldsymbol{i}$ = 1,2..., ***n,*** виробляючи проміжне значення ***g' .***
- Далі по конвеєру процесорів йде обмін значень ***g'.*** Процесори кумулятивно обчислюють $g(x_1, x_2,\ldots,x_n)$ за n-1 кроків: P_i, отримує ***g'*** від P_{i-1} і посилає ***g''*** до $\boldsymbol{P_{i+1}}$, для $2 \le \boldsymbol{i < n} - 1$, або на вихід, якщо $\boldsymbol{i = n}$.
- Отже, $\boldsymbol{T_n = n}$.

Прискорення і уповільнення

- Прискорення $\boldsymbol{s(l,n)= ((n-1)2^n + n)/n,}$ є суперлінійним по $\boldsymbol{n}$ і, отже, суперечить теоремі про прискорення.
- Якщо ж на $\boldsymbol{P}$ доступні тільки q процесорів$\boldsymbol{,}$ $2 \le q < \boldsymbol{n,}$ то тільки q з $\boldsymbol{n}$ змінних (наприклад, $x_1, x_2,...,x_q$) читаються безпосередньо з вхідного буфера (один – кожним процесором).
- Тим часом, значення решти $\boldsymbol{n\text{-}q}$ змінних зникають і повинні бути відновлені з $\boldsymbol{f(}x_1, x_2,...,x_n\boldsymbol{)}$. Отже, $\boldsymbol{T_q = .}$ Тому, $\boldsymbol{d(q,n)=} T_q/T_n$ є суперлінійним по і, отже, знову теорема про уповільнення не виконується.

Висновки

- Отже, в режимі *реального часу* обчислення на паралельному комп'ютері можна істотно поліпшити показники продуктивності в порівнянні з виконанням на послідовного комп'ютері.
- Це поліпшення у швидкості обчислень може бути *суперлінійним* за кількістю процесорів, використаних паралельним комп'ютером.
- В основі цього результату – той факт, що при обчисленнях в реальному часі (з обмеженнями на крайні терміни виконання) ***принцип моделювання вже не діє***.
- Тобто, твердження про те, що обчислення, що виконуються паралельно, можуть так чи інакше бути відтворені в послідовному режимі з тими же результатами відносно швидкості в контексті реального часу втрачає сенс і не справджується.